

# Hospital Management Database Setup

--

=====

-- HOSPITAL MANAGEMENT DATABASE

=====

CREATE DATABASE hospital\_db;

USE hospital\_db;

--

=====

-- PATIENTS

--

=====

```
CREATE TABLE patients (  
    patient_id INT PRIMARY KEY,  
    patient_name VARCHAR(100),  
    gender VARCHAR(10),  
    age INT,  
    city VARCHAR(50)  
);
```

```
INSERT INTO patients VALUES  
(1,'Rahul','Male',35,'Chennai'),  
(2,'Priya','Female',28,'Bangalore'),
```

(3,'Arun','Male',45,'Hyderabad'),  
(4,'Sneha','Female',31,'Mumbai'),  
(5,'Kiran','Male',50,'Delhi'),  
(6,'Ramesh Kumar','Male',42,'Chennai'),  
(7,'Lakshmi Devi','Female',36,'Coimbatore'),  
(8,'Suresh Babu','Male',29,'Madurai'),  
(9,'Anitha Rao','Female',55,'Bangalore'),  
(10,'Vijay Sharma','Male',61,'Mumbai'),  
(11,'Deepa Nair','Female',48,'Kochi'),  
(12,'Karthik Raj','Male',34,'Chennai'),  
(13,'Meena Patel','Female',40,'Ahmedabad'),  
(14,'Rohit Singh','Male',27,'Delhi'),  
(15,'Pooja Gupta','Female',31,'Hyderabad'),  
(16,'Naveen Kumar','Male',45,'Pune'),  
(17,'Shalini Verma','Female',52,'Lucknow'),  
(18,'Ajay Das','Male',38,'Kolkata'),  
(19,'Neha Reddy','Female',24,'Hyderabad'),  
(20,'Manoj Jain','Male',58,'Jaipur');

--

=====

-- DOCTORS

--

=====

```
CREATE TABLE doctors (  
    doctor_id INT PRIMARY KEY,  
    doctor_name VARCHAR(100),  
    specialization VARCHAR(50),
```

```
consultation_fee DECIMAL(10,2)
);
```

```
INSERT INTO doctors VALUES
(101,'Dr. Kumar','Cardiology',1000),
(102,'Dr. Mehta','Neurology',1500),
(103,'Dr. Sharma','Orthopedics',1200),
(104,'Dr. Reddy','Dermatology',800),
(105,'Dr. Anjali Iyer','Pediatrics',900),
(106,'Dr. Rajesh Gupta','ENT',850),
(107,'Dr. Vivek Menon','General Medicine',700),
(108,'Dr. Sunita Shah','Gynecology',1300),
(109,'Dr. Akash Verma','Cardiology',1200),
(110,'Dr. Harish Rao','Neurology',1800),
(111,'Dr. Kavitha Devi','Orthopedics',1400),
(112,'Dr. Prakash Nair','Pulmonology',1100);
```

```
--
```

```
=====
```

```
-- APPOINTMENTS
```

```
--
```

```
=====
```

```
CREATE TABLE appointments (
    appointment_id INT PRIMARY KEY,
    patient_id INT,
    doctor_id INT,
    appointment_date DATE,
    status VARCHAR(20),
```

```
FOREIGN KEY(patient_id)
REFERENCES patients(patient_id),

FOREIGN KEY(doctor_id)
REFERENCES doctors(doctor_id)
);
```

```
INSERT INTO appointments VALUES
(1001,1,101,'2025-05-01','Completed'),
(1002,2,102,'2025-05-03','Completed'),
(1003,3,101,'2025-05-05','Pending'),
(1004,4,103,'2025-05-06','Completed'),
(1005,5,104,'2025-05-08','Cancelled'),
(1006,6,105,'2025-05-10','Completed'),
(1007,7,106,'2025-05-11','Completed'),
(1008,8,107,'2025-05-11','Completed'),
(1009,9,108,'2025-05-12','Pending'),
(1010,10,109,'2025-05-12','Completed'),
(1011,11,110,'2025-05-13','Completed'),
(1012,12,111,'2025-05-13','Completed'),
(1013,13,112,'2025-05-14','Cancelled'),
(1014,14,105,'2025-05-14','Completed'),
(1015,15,106,'2025-05-15','Completed'),
(1016,16,107,'2025-05-16','Pending'),
(1017,17,108,'2025-05-17','Completed'),
(1018,18,109,'2025-05-17','Completed'),
(1019,19,110,'2025-05-18','Completed'),
(1020,20,111,'2025-05-18','Pending'),
```

```
(1021,1,112,'2025-05-19','Completed'),  
(1022,2,105,'2025-05-20','Completed'),  
(1023,3,106,'2025-05-20','Completed'),  
(1024,4,107,'2025-05-21','Completed'),  
(1025,5,108,'2025-05-21','Cancelled');
```

```
--
```

```
=====
```

```
-- BILLING
```

```
--
```

```
=====
```

```
CREATE TABLE billing (  
    bill_id INT PRIMARY KEY,  
    appointment_id INT,  
    bill_amount DECIMAL(10,2),  
    payment_status VARCHAR(20),  
  
    FOREIGN KEY(appointment_id)  
        REFERENCES appointments(appointment_id)  
);
```

```
INSERT INTO billing VALUES  
(1,1001,1000,'Paid'),  
(2,1002,1500,'Paid'),  
(3,1003,1000,'Pending'),  
(4,1004,1200,'Paid'),  
(5,1005,0,'Cancelled'),  
(6,1006,900,'Paid'),
```

```
(7,1007,850,'Paid'),
(8,1008,700,'Paid'),
(9,1009,1300,'Pending'),
(10,1010,1200,'Paid'),
(11,1011,1800,'Paid'),
(12,1012,1400,'Paid'),
(13,1013,0,'Cancelled'),
(14,1014,900,'Paid'),
(15,1015,850,'Paid'),
(16,1016,700,'Pending'),
(17,1017,1300,'Paid'),
(18,1018,1200,'Paid'),
(19,1019,1800,'Paid'),
(20,1020,1400,'Pending'),
(21,1021,1100,'Paid'),
(22,1022,900,'Paid'),
(23,1023,850,'Paid'),
(24,1024,700,'Paid'),
(25,1025,0,'Cancelled');
```

```
--
```

```
=====
```

```
-- TREATMENTS
```

```
--
```

```
=====
```

```
CREATE TABLE treatments (
    treatment_id INT PRIMARY KEY,
    appointment_id INT,
```

```
treatment_name VARCHAR(100),
treatment_cost DECIMAL(10,2),

FOREIGN KEY(appointment_id)
REFERENCES appointments(appointment_id)
);
```

```
INSERT INTO treatments VALUES
(1,1001,'ECG',500),
(2,1002,'Brain Scan',2500),
(3,1004,'Knee X-Ray',1200),
(4,1006,'Vaccination',300),
(5,1007,'Ear Cleaning',400),
(6,1008,'General Checkup',250),
(7,1010,'ECG',500),
(8,1011,'MRI Scan',3500),
(9,1012,'Bone Scan',1800),
(10,1014,'Vaccination',300),
(11,1015,'ENT Checkup',500),
(12,1017,'Pregnancy Scan',2200),
(13,1018,'Stress Test',900),
(14,1019,'Neurology Test',4000),
(15,1021,'Pulmonary Function Test',1600);
```

```
--
```

```
=====
```

```
-- MEDICINES
```

```
--
```

```
=====
```

```
CREATE TABLE medicines (  
    medicine_id INT PRIMARY KEY,  
    medicine_name VARCHAR(100),  
    medicine_price DECIMAL(10,2)  
);
```

```
INSERT INTO medicines VALUES  
(1,'Paracetamol',20),  
(2,'Amoxicillin',120),  
(3,'Aspirin',50),  
(4,'Metformin',180),  
(5,'Atorvastatin',250),  
(6,'Omeprazole',90),  
(7,'Cetirizine',40),  
(8,'Vitamin D',150);
```

```
--
```

```
=====
```

```
-- PRESCRIPTIONS
```

```
--
```

```
=====
```

```
CREATE TABLE prescriptions (  
    prescription_id INT PRIMARY KEY,  
    appointment_id INT,  
    medicine_id INT,  
    quantity INT,
```

```
FOREIGN KEY(appointment_id)
REFERENCES appointments(appointment_id),

FOREIGN KEY(medicine_id)
REFERENCES medicines(medicine_id)
);
```

```
INSERT INTO prescriptions VALUES
(1,1001,1,10),
(2,1001,3,5),
(3,1002,2,7),
(4,1004,5,15),
(5,1006,7,10),
(6,1007,6,5),
(7,1008,1,10),
(8,1010,4,20),
(9,1011,5,30),
(10,1012,3,15),
(11,1014,8,10),
(12,1015,7,5),
(13,1017,2,10),
(14,1018,1,20),
(15,1019,5,15);
```

```
--
```

```
=====
```

```
-- APPOINTMENT AUDIT
```

```
--
```

```
=====
```

```
CREATE TABLE appointment_audit (  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    appointment_id INT,  
    action_type VARCHAR(50),  
    action_time TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
);
```

```
--
```

```
=====
```

```
-- DATA SUMMARY
```

```
--
```

```
=====
```

```
-- Patients      : 20
```

```
-- Doctors       : 12
```

```
-- Appointments  : 25
```

```
-- Billing        : 25
```

```
-- Treatments    : 15
```

```
-- Medicines     : 8
```

```
-- Prescriptions : 15
```

```
-- Audit Table   : Empty (for Trigger Practice)
```

```
--
```

```
=====
```

# . FUNCTIONS

---

## Definition

A Function is a database object that accepts input, performs calculations, and returns a single value.

---

## When to Use

- ✓ Tax calculation
  - ✓ Discount calculation
  - ✓ Age calculation
  - ✓ Salary calculation
  - ✓ Reusable business logic
- 

## Syntax

```
CREATE FUNCTION function_name(parameters)
RETURNS datatype
DETERMINISTIC
BEGIN
```

```
logic
```

RETURN value;

END;

---

## Example

Calculate GST

DELIMITER \$\$

```
CREATE FUNCTION calculate_gst(amount  
DECIMAL(10,2))  
RETURNS DECIMAL(10,2)
```

DETERMINISTIC

BEGIN

    RETURN amount \* 0.18;

END \$\$

DELIMITER ;

Usage

```
SELECT calculate_gst(1000);
```

Output

## Types of Functions

### Built-in Functions

COUNT()

SUM()

AVG()

MAX()

MIN()

UPPER()

LOWER()

---

### User Defined Functions (UDF)

Created by users.

CREATE FUNCTION

---

## Common Mistakes

- ✗ Missing RETURN
- ✗ Wrong return datatype
- ✗ Multiple return values
- ✗ Using function for large transactions

---

# Function Exercises

## Easy

1. Create function to calculate doctor's tax (18%).
  2. Create function to calculate appointment fee after 10% discount.
  3. Create function to convert patient name to uppercase.
- 

## Medium

4. Create function to classify patient age.

<18 Child

18-60 Adult

>60 Senior

5. Create function to calculate days since appointment.
- 

## Difficult

6. Create function returning total billing amount for a patient.
  7. Create function returning doctor's total appointments.
-

## 2. STORED PROCEDURES

---

### Definition

A Stored Procedure is a group of SQL statements stored in the database and executed as a unit.

Pre computed /calculated storage

---

### When to Use

- ✓ Reports
  - ✓ Batch updates
  - ✓ Data processing
  - ✓ ETL jobs
  - ✓ Complex business logic
- 

### Syntax

DELIMITER \$\$

CREATE PROCEDURE procedure\_name()

BEGIN

SQL statements

END \$\$

DELIMITER ;

---

## **Example**

Display all patients.

DELIMITER \$\$

CREATE PROCEDURE get\_patients()

BEGIN

SELECT \* FROM patients;

END \$\$

DELIMITER ;

Execute

CALL get\_patients();

---

## Procedure with Input Parameter

DELIMITER \$\$

```
CREATE PROCEDURE get_patient_by_id(  
    IN p_id INT  
)  
BEGIN
```

```
SELECT *  
FROM patients  
WHERE patient_id = p_id;
```

```
END $$
```

DELIMITER ;

Execute

```
CALL get_patient_by_id(1);
```

---

## Procedure Types

### No Parameter

```
CALL procedure();
```

---

### Input Parameter

IN

---

## Output Parameter

OUT

---

## Input Output Parameter

INOUT

---

## Common Mistakes

- ✗ Forgetting DELIMITER
  - ✗ Confusing function with procedure
  - ✗ Using RETURN instead of OUT
  - ✗ Not handling NULL values
- 

## Stored Procedure Exercises

### Easy

1. Display all doctors.
  2. Display all appointments.
  3. Get patient details using patient id.
- 

### Medium

4. Get appointments by doctor id.
  5. Display all completed appointments.
  6. Display total revenue.
- 

#### Difficult

7. Create procedure to insert new patient.
  8. Create procedure to generate doctor-wise revenue report.
  9. Create procedure showing top consulted doctor.
- 

## 3. VIEWS

---

### Definition

A View is a virtual table based on a SQL query.

---

### When to Use

- ☒ Security
- ☒ Reporting
- ☒ Simplifying joins
- ☒ Restricting access

---

## Syntax

```
CREATE VIEW view_name AS
```

```
SELECT ...
```

---

## Example

```
CREATE VIEW patient_appointments AS
```

```
SELECT  
p.patient_name,  
d.doctor_name,  
a.appointment_date,  
a.status
```

```
FROM patients p  
JOIN appointments a  
ON p.patient_id=a.patient_id
```

```
JOIN doctors d  
ON a.doctor_id=d.doctor_id;
```

Usage

```
SELECT * FROM patient_appointments;
```

---

# Types of Views

## Simple View

One table

## Complex View

Multiple tables

## Updatable View

Allows DML

## Read Only View

Only SELECT

---

## Common Mistakes

- ✗ Treating view as physical table
  - ✗ Updating non-updatable views
  - ✗ Excessive nested views
- 

## View Exercises

Easy

1. Create view showing doctor details.

2. Create view showing patient details.

---

#### Medium

3. Create view for completed appointments.

4. Create view for paid bills.

---

#### Difficult

5. Create doctor revenue view.

6. Create patient history view.

---

## 4. TRIGGERS

---

### Definition

Trigger automatically executes when INSERT, UPDATE, DELETE occurs.

---

### When to Use

☒ Auditing

☒ Logging

✓ Validation

✓ History tracking

---

## Syntax

CREATE TRIGGER trigger\_name

BEFORE/AFTER

INSERT/UPDATE/DELETE

ON table\_name

FOR EACH ROW

BEGIN

logic

END;

---

## Example

Audit appointment inserts.

DELIMITER \$\$

```
CREATE TRIGGER trg_appointment_insert
```

```
AFTER INSERT
```

```
ON appointments
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO appointment_audit
```

```
(  
  appointment_id,  
  action_type  
)
```

```
VALUES
```

```
(  
  NEW.appointment_id,  
  'INSERT'  
);
```

```
END $$
```

```
DELIMITER ;
```

```
=====Trigger for deletion =====  
drop trigger trg_appointment_delete;
```

```
DELIMITER $$
CREATE TRIGGER trg_appointment_delete
before delete
ON appointments
FOR EACH ROW
BEGIN
INSERT INTO appointment_audit
(
appointment_id,action_type
)

VALUES
(
old.appointment_id,
'DELETE'
);

END $$

DELIMITER ;
```

---

## Test

```
INSERT INTO appointments
```

```
VALUES  
(  
  1006,  
  1,  
  102,  
  '2025-06-01',  
  'Pending'  
);
```

Audit record automatically inserted.

---

## Types

**BEFORE INSERT**

**AFTER INSERT**

**BEFORE UPDATE**

**AFTER UPDATE**

**BEFORE DELETE**

**AFTER DELETE**

---

## Common Mistakes

 Recursive trigger logic

- ✗ Performance issues
  - ✗ Business logic overload
  - ✗ Forgetting OLD and NEW keywords
- 

## Trigger Exercises

### Easy

1. Audit new patients.
  2. Audit deleted appointments.
- 

### Medium

3. Log appointment status changes.
  4. Track doctor fee updates.
- 

### Difficult

5. Prevent appointment insertion if status is NULL.
  6. Prevent billing amount below zero.
- 

## 5. CTE (Common Table Expression)

---

## Definition

A temporary named result set used within a query.

---

## When to Use

- ✓ Complex SQL
  - ✓ Replace subqueries
  - ✓ Recursive hierarchy
  - ✓ Better readability
- 

## Syntax

```
WITH cte_name AS  
(  
    SELECT ...  
)
```

```
SELECT *  
FROM cte_name;
```

---

## Example

Appointments above average billing.

```
WITH avg_bill AS
(
    SELECT AVG(bill_amount) avg_amount
    FROM billing
)

SELECT *
FROM billing

WHERE bill_amount >
(
    SELECT avg_amount
    FROM avg_bill
);
```

---

## **Recursive CTE**

Used for hierarchy.

Example:

```
WITH RECURSIVE numbers AS
(
    SELECT 1 num

    UNION ALL

    SELECT num+1
    FROM numbers
```

```
WHERE num < 10  
)
```

```
SELECT *  
FROM numbers;
```

---

## Common Mistakes

- ✗ Forgetting WITH
  - ✗ Missing recursive termination
  - ✗ Using CTE repeatedly in huge queries
  - ✗ Confusing CTE with View
- 

## CTE Exercises

### Easy

1. Find bills above average.
  2. Find doctors with above-average consultation fee.
  3. Find patients older than average age.
- 

### Medium

4. Find top revenue-generating doctors using CTE.
5. Find appointment count by doctor using CTE.

6. Find patient visit count using CTE.

---

**Difficult**

7. Find doctor contributing highest revenue.

8. Find patient with maximum billing amount.

9. Use recursive CTE to generate numbers 1-20.

---

## Interview Comparison Cheat Sheet

| Feature             | Function | Procedure | View          | Trigger     | CTE              |
|---------------------|----------|-----------|---------------|-------------|------------------|
| Returns Value       | Yes      | Optional  | Virtual Table | No          | Temporary Result |
| Accept Parameters   | Yes      | Yes       | No            | No          | No               |
| Used in SELECT      | Yes      | No        | Yes           | No          | Yes              |
| Automatic Execution | No       | No        | No            | Yes         | No               |
| Stores Logic        | Yes      | Yes       | Query         | Event Logic | Query Logic      |

|          |              |                  |           |          |                 |
|----------|--------------|------------------|-----------|----------|-----------------|
| Best Use | Calculations | Business Process | Reporting | Auditing | Complex Queries |
|----------|--------------|------------------|-----------|----------|-----------------|

# 1. Function vs Procedure

## Definition

### Function

- Accepts parameters
- Returns exactly one value
- Used mainly for calculations
- No DML
- Select

### Procedure

DML operations can be performed  
 Call sp ;

- Executes a set of SQL statements
- May return zero, one, or multiple values
- Used for business processes

---

## Comparison

| Feature | Function | Procedure |
|---------|----------|-----------|
|---------|----------|-----------|

|                            |              |                  |
|----------------------------|--------------|------------------|
| Returns Value              | Mandatory    | Optional         |
| Called Using               | SELECT       | CALL             |
| Used in SELECT             | Yes          | No               |
| Can Return Multiple Values | No           | Yes (OUT params) |
| Best For                   | Calculations | Business Logic   |

---

## Example

### Function

```
SELECT calculate_tax(1000);
```

Returns:

180

### Procedure

```
CALL generate_revenue_report();
```

Returns result set.

---

## Interview Answer

Functions are primarily used for reusable calculations and must return a value, whereas procedures are used for executing a sequence of business operations and may or may not return values.

---

## 2. View vs CTE

### View

A stored query saved permanently in the database.

```
CREATE VIEW doctor_revenue AS
SELECT doctor_id,
       SUM(bill_amount)
FROM billing
GROUP BY doctor_id;
```

---

### CTE

Temporary query valid only during execution.

```
WITH revenue AS
(
  SELECT doctor_id,
         SUM(bill_amount) total
  FROM billing
  GROUP BY doctor_id
)
```

```
SELECT *  
FROM revenue;
```

---

## Comparison

| Feature           | View      | CTE                  |
|-------------------|-----------|----------------------|
| Stored in DB      | Yes       | No                   |
| Reusable          | Yes       | No                   |
| Lifetime          | Permanent | Query execution only |
| Recursive Support | No        | Yes                  |
| Maintenance       | Easy      | Temporary            |

---

## When to Use

### View

Use when:

Daily reports  
Dashboards  
Security

Reusable queries

**CTE**

Use when:

Complex query

Recursive hierarchy

Replace nested subqueries

Improve readability

---

### **Interview Answer**

A View is a permanent virtual table stored in the database, whereas a CTE is a temporary result set used only within a single query.

---

## **3. Trigger vs Procedure**

### **Trigger**

Runs automatically.

### **Procedure**

Runs manually.

---

### **Example**

**Trigger**

INSERT INTO appointments ...

Automatically logs audit entry.

No CALL required.

---

#### **Procedure**

CALL insert\_patient();

Must be executed explicitly.

---

#### **Comparison**

| <b>Feature</b>       | <b>Trig<br/>ger</b> | <b>Procedu<br/>re</b> |
|----------------------|---------------------|-----------------------|
| Automatic            | Yes                 | No                    |
| Event<br>Driven      | Yes                 | No                    |
| Called<br>Explicitly | No                  | Yes                   |
| Best For             | Audit<br>ing        | Business<br>Logic     |

---

#### **Interview Answer**

A Trigger executes automatically when a database event occurs, whereas a Procedure executes only when explicitly called.

---

## 4. Why Use CTE Instead of Subquery?

### Subquery Example

```
SELECT *  
FROM billing  
WHERE bill_amount >  
(  
    SELECT AVG(bill_amount)  
    FROM billing  
);
```

Simple and acceptable.

---

### Complex Subquery Problem

```
SELECT ...  
(  
    SELECT ...  
(  
        SELECT ...  
    ))
```

Hard to read.

---

## **CTE Solution**

```
WITH avg_bill AS  
(  
  SELECT AVG(bill_amount) avg_amt  
  FROM billing  
)
```

```
SELECT *  
FROM billing  
WHERE bill_amount >  
(  
  SELECT avg_amt  
  FROM avg_bill  
);
```

---

## **Advantages of CTE**

**Readability**

**Reusability**

**Easier Debugging**

**Recursive Support**

**Modular SQL**

---

## **Interview Answer**

CTEs improve readability and maintainability of complex SQL queries and are especially useful when the same intermediate result is used multiple times.

---

## 5. BEFORE vs AFTER Trigger

---

### **BEFORE Trigger**

Executes before data modification.

BEFORE INSERT  
BEFORE UPDATE  
BEFORE DELETE

---

### **Usage**

Validation

Example:

Prevent negative bill amount.

```
IF NEW.bill_amount < 0
THEN
    SIGNAL ERROR;
END IF;
```

---

## **AFTER Trigger**

Executes after modification.

AFTER INSERT

AFTER UPDATE

AFTER DELETE

---

## **Usage**

Audit Logging

AFTER INSERT

Insert record into audit table.

---

## **Comparison**

### **BEFORE**

Before  
action

Validation

Modify NEW  
values

### **AFTER**

After action

Logging

Cannot prevent  
completed action

---

## **Interview Answer**

BEFORE triggers are used for validation and data modification before saving, while AFTER triggers are mainly used for logging and auditing after data is successfully saved.

---

## 6. NEW vs OLD in Triggers

Used inside UPDATE/DELETE triggers.

---

### NEW

Represents new incoming values.

NEW.salary

NEW.status

---

### OLD

Represents existing values.

OLD.salary

OLD.status

---

### Example

Status Change

Before:

Pending

After:

Completed

Trigger:

OLD.status

Returns:

Pending

NEW.status

Returns:

Completed

---

## **Usage**

Audit changes.

Patient Status Changed

Salary Changed

Fee Changed

---

## **Interview Answer**

NEW refers to the new value being inserted or updated, while OLD refers to the previous value before the change.

---

## 7. Recursive CTE Use Cases

### What is Recursive CTE?

A CTE that refers to itself.

---

### Syntax

```
WITH RECURSIVE cte_name AS  
(  
  Anchor Query  
  
  UNION ALL  
  
  Recursive Query  
)
```

---

### Example

Generate Numbers

```
WITH RECURSIVE nums AS  
(  
  SELECT 1  
  
  UNION ALL
```

```
SELECT num + 1  
FROM nums  
WHERE num < 10  
)
```

```
SELECT *  
FROM nums;
```

---

## **Real-Time Use Cases**

### **Employee Hierarchy**

CEO



Manager



Employee

---

### **Category Hierarchy**

Electronics



Mobiles



Smartphones

---

### **Organization Structure**

**Folder Structure**

**Bill of Materials (Manufacturing)**

---

## **Interview Answer**

Recursive CTEs are used for hierarchical and parent-child relationships such as employee-manager hierarchies, folder structures, and organizational charts.

---

## **8. Updatable vs Non-Updatable View**

---

### **Updatable View**

Can perform:

INSERT  
UPDATE  
DELETE

Usually based on a single table.

Example:

```
CREATE VIEW patient_view AS
```

```
SELECT *  
FROM patients;
```

---

### **Non-Updatable View**

Usually contains:

JOIN

GROUP BY

DISTINCT

UNION

Aggregate Functions

Example:

```
CREATE VIEW doctor_revenue AS
```

```
SELECT doctor_id,  
       SUM(bill_amount)  
FROM billing  
GROUP BY doctor_id;
```

Cannot update.

---

## **Interview Answer**

An updatable view allows DML operations because it maps directly to a single base table, while a non-updatable view contains joins, aggregates, or grouping that prevent direct updates.

---

## **9. IN vs OUT Parameters**

Used in Stored Procedures.

---

### **IN Parameter**

Input only.

```
CREATE PROCEDURE get_patient  
(  
  IN p_id INT  
)
```

Call:

```
CALL get_patient(1);
```

---

### **OUT Parameter**

Output only.

```
CREATE PROCEDURE get_count  
(  
  OUT total INT  
)
```

Procedure assigns value.

---

### **Example**

```
SET total = 25;
```

---

## Comparison

| IN             | OUT               |
|----------------|-------------------|
| Sends value in | Returns value out |
| Read-only      | Write-only        |
| Used for input | Used for output   |

---

## INOUT

Acts as both.

INOUT amount

---

## Interview Answer

IN parameters pass values into a procedure, OUT parameters return values from a procedure, and INOUT parameters do both.

---

## 10. Can a Function Call a Procedure?

### Short Answer

Generally No

MySQL functions are restricted.

---

## Why?

Functions are expected to:

Be deterministic

Return a value

Avoid side effects

Procedures may:

Insert data

Delete data

Commit transactions

Rollback transactions

This would make functions unsafe.

---

## Example

 Not allowed:

```
CREATE FUNCTION test()  
RETURNS INT
```

```
BEGIN
```

```
CALL my_procedure();
```

RETURN 1;

END;

---

### **What is Allowed?**

Procedure can call Function

CALL generate\_bill();

Inside:

SET tax = calculate\_tax(amount);

Perfectly valid.

---

### **Interview Answer**

In MySQL, functions generally cannot call stored procedures because procedures may perform operations that violate function restrictions. However, procedures can call functions without issue.

---

## **Quick Interview Summary**

**Question**

**One-Line Answer**

|                          |                                                             |
|--------------------------|-------------------------------------------------------------|
| Function vs Procedure    | Function returns a value; Procedure executes business logic |
| View vs CTE              | View is permanent; CTE is temporary                         |
| Trigger vs Procedure     | Trigger is automatic; Procedure is manual                   |
| Why CTE?                 | Better readability and recursive support                    |
| BEFORE vs AFTER          | Validation vs Auditing                                      |
| NEW vs OLD               | New values vs Previous values                               |
| Recursive CTE            | Hierarchical data processing                                |
| Updatable View           | Single-table view allowing DML                              |
| IN vs OUT                | Input parameter vs Output parameter                         |
| Function Call Procedure? | Generally No in MySQL                                       |